

**CS102 – Algorithms and Programming II**  
**Programming Assignment 4**  
**Fall 2025-2026**

**ATTENTION:**

- Compress all of the Java program source files (.java) files into a single zip file.
- The name of the zip file should follow the below convention, where you replace the variables “Sec1”, “Surname” and “Name” with your actual section, surname and name:  
**CS102\_Sec1\_Asst4\_Surname\_Name.zip**
- Upload the above zip file to Moodle by the deadline (if not, significant points will be taken off). You will get a chance to update and improve your solution by consulting to the TAs and tutors during the lab. You have to make the preliminary submission before the lab day. After the TA checks your work in the lab, you will make your final submission. Even if your code does not change in between these two versions, you should make two submissions for each assignment.

**GRADING WARNING:**

- Please read the grading criteria provided on Moodle. The work must be done individually. Code sharing is strictly forbidden. We are using sophisticated tools to check the code similarities. The Honor Code specifies what you can and cannot do. Breaking the rules will result in disciplinary action.

**EXTENDING BOARD GAME WITH INTERACTIVE GRAPHICS AND CONTROLS**

For this lab assignment, you will implement a Java GUI application that extends your simple board game from the first lab with interactive graphics and mouse controls. You are expected to use Java Swing or AWT library to implement the GUI; if you prefer, you can also use JavaFX. The application should consist of two main screens. In the main menu, we should select the number of players and have text boxes to enter each player’s name. The game screen should include the main interaction with the game board, allowing players to move on the grid based on the rules introduced in the first lab. You will represent the empty cells, walls, and pickable objects with different colored rectangles. You will also represent each player with a different colored circle.

All the interactions should be done through the GUI; we should not enter anything into the console. At the beginning, display the menu so the user can choose the number of players. Include buttons with the options 2, 3, and 4.

| Menu              |   |   |
|-------------------|---|---|
| How many players? |   |   |
| 2                 | 3 | 4 |

To give functionality to a button, we use `ActionListeners`. Suppose we have a `JButton` that we want to print on the console when we click on it. You may use the following shorthand to attach a new `ActionListener` to this button with its `actionPerformed` method overridden:

```

JButton button = new JButton("My Button");
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Hey!");
    }
});

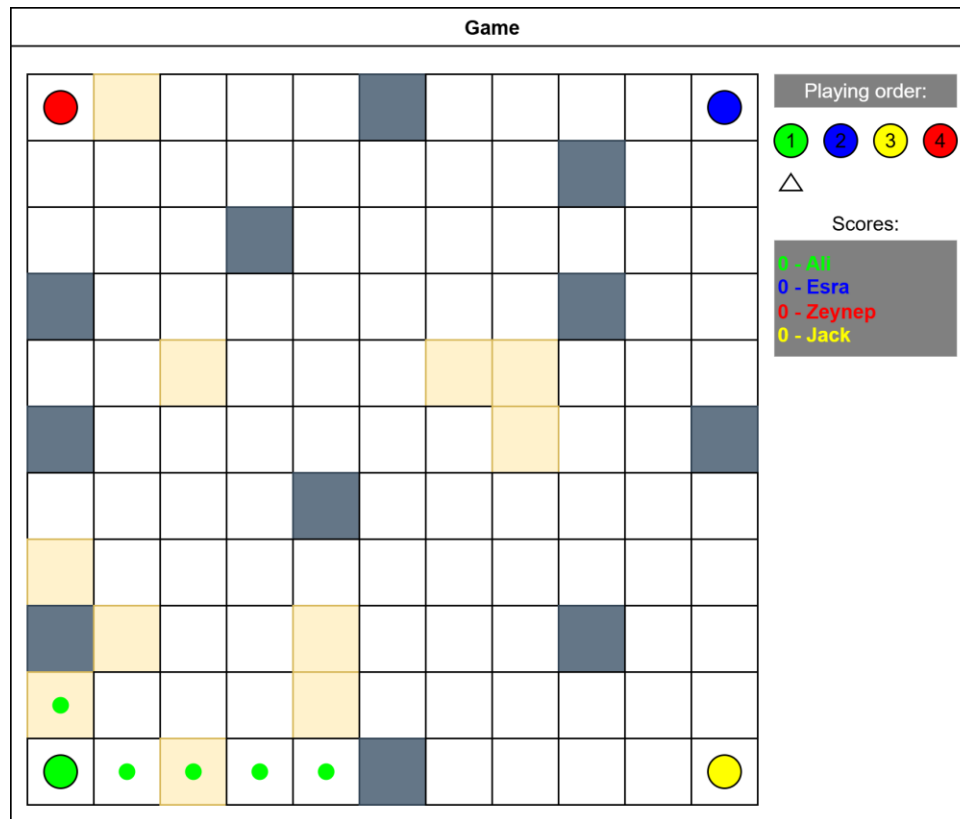
```

You should also add this button to a frame or a panel to be able to use it. After the user chooses the number of players, you will create a new frame for the user to enter the players' names. The frame should contain text fields based on the number of players selected. For example, if we choose 2, there should be two inputs for the player names. In this example, we display a version of the Player Names screen for four players.

| Player Names                                                                        |                                      |
|-------------------------------------------------------------------------------------|--------------------------------------|
| Please enter player names:                                                          |                                      |
|   | <input type="text"/>                 |
|  | <input type="text"/>                 |
|  | <input type="text"/>                 |
|  | <input type="text"/>                 |
| <input type="button" value="Back to Menu"/>                                         | <input type="button" value="Start"/> |

If the user does not enter one of the names, clicking the Start button should display an error message saying the user should enter a name for each player. You should also check if all the names are unique. Display a different error message if some players have the same name. The start button will proceed to the Game screen, given that we have a unique name for all players. The Back to Menu button should close this window to return to the previous screen.

The Game screen includes the map, the player order, and the score table. You may refer to the following mockup for the design. You are encouraged to use images instead of colored blocks to represent empty cells, the walls, and the pickable objects. You should have a distinct color or image for each player. You should display the randomly determined player order in the corner, with an indicator of the current player. For example, our design uses a small triangle below the current player's color.



You should also display the players' current scores. This list should indicate the player's color and name, as well as their current score. This list should be in descending order. For the current player, include an indicator for the reachable cells on the map. For example, in the image above, the small green circles represent the cells the current player can move into. The user will click on the related cell to move his/her piece into. If it is a cell that includes a pickable object (yellow cells in the example), the cell turns empty, and the player moves into that cell. Similarly, if another player occupies the reachable cell, that player is captured. If the user clicks on an unreachable cell (out of range or a wall), display a message indicating this is not allowed.

You may use the following code to show a one-time pop-up message for such errors:

```
JOptionPane.showMessageDialog(null, "My Message!");
```

After clicking the OK button, we will return to the previous view using such message dialogues, which are useful for such errors or warnings.

Once the game ends, display a new frame with the final scores. You should also write the name(s) of the winner. Include options to play again with the same player count and names, or return to the initial menu where we choose a new player count.

**Preliminary Submission:** You will submit an early version of your solution before the final submission. This version should at least include the following:

- The map screen and player logic should be complete, with players taking turns to move and pop-up error messages on incorrect moves.

**Not completing the preliminary submission on time results in 50% reduction of this assignment's final grade.**